Assignment 3

Due: Tuesday, 10th April, 2026 by 17:00 (*UTC +7*)
Points: 45
Submitting: (a) file upload (*PDF, zip + other files in the submission requirements*)
Due Dates: 10 April 2026 at 17:00 pm (UTC+7)
Available: 1 March at 0:00 - 10 April at 20:00

Assignment Name	**Full-Stack Web Development for AI Application in Cybersecurity Scenarios**
Assignment Description	In this final assignment, students will develop a full-stack web application that integrates the AI model created in Assignment 2. Using React.js for the front end and FastAPI for the back end, students will build an application that allows users to input data, processes the input using the AI model on the server side, and displays the results through interactive visualizations on the front-end. As the culminating assignment of the course, this project carries the largest weight in the final grade. It will test students' ability to integrate front-end and back-end technologies, handle data efficiently, and deliver clear visualizations of the AI model's output. In this assignment, students are expected to demonstrate proficiency in the following areas: ● **Full-Stack Web Development:** Implement a complete web application using React.js for the user interface and FastAPI for server-side processing. The front end must capture user inputs, and the back end must handle the AI model's processing and return the prediction results. ● **AI Model Integration:** Connect the AI model to the FastAPI back-end, enabling real-time predictions based on user input data. This will involve passing data between the front-end and back-end and efficiently processing it with the AI model. ● **Data Visualization:** Use React.js along with visualization libraries like D3.js, Chart.js, or Plotly.js to present the AI model's predictions. Ensure the visualizations are dynamic and provide clear, meaningful insights into the model's output. At least two types of charts must be included. ● **Error Handling and Input Validation:** Ensure robust error handling and input validation in both the front-end and back-end. The front end should validate user inputs before sending them to the server, and the back end must handle any errors gracefully, providing appropriate feedback to the user.

	Depending on the AI model's topic, students' applications might focus on one of the following areas: 1. **Misinformation Detection on Social Media:** Detect misinformation on social media platforms. Students will collect real-world social media data, extract useful features (like text content and user behaviour), and train models to identify false or misleading posts. The goal is to help improve online safety and address a key cybersecurity challenge in today's digital world. 2. **Spam or Malware Detection:** Detect spam messages or potential malware threats. Students will work with datasets containing emails, messages, or files, extract relevant features (such as text patterns or file properties), and build models to classify harmful content. The project aims to help protect users from digital threats and improve cybersecurity in communication systems. 3. **Detecting Software Vulnerabilities:** Detect security vulnerabilities in software code. Students will analyse source code to identify patterns that may lead to bugs or exploits, and train models to automatically flag risky code. The goal is to help developers find and fix vulnerabilities early, improving the overall security of software systems. 4. **Network Traffic Classification for Anomaly Detection:** Classify network traffic and detect unusual or potentially malicious activity. Students will analyse network data, extract features such as packet size and timing, and train models to distinguish normal behaviour from anomalies. The goal is to support early detection of cyberattacks and improve network security. 5. **Cybersecurity threat detection in banking:** This covers a range of cybersecurity problems in the banking sector. Students are expected to do some research to choose a suitable cybersecurity problem (e.g. fraud detection, identify theft, etc.) to solve using AI engineering. The goal is to improve the system security in this important industry sector.
Weight	45% of your total marks for the unit
Due Date	Tuesday, 10th April, 2026, 5:00 pm (17:00) UTC +7

Submission	**1.** Report: ● Size: No more than 8 pages (*excluding cover page, table of contents, bibliography, and appendix*) ● Format: PDF **2.** Source Code: ● Format: ZIP file. ● Contents: ➢ Front-end React.js code (excluding node_modules) ➢ Back-end FastAPI code (must include the AI model code used in the web application) ➢ README file (Markdown or PDF format) **3.** Video Demonstration: ● Format: MP4 or AVI (maximum 7 minutes). **4.** Meeting Minutes: ● Format: PDF. **5.** Contribution Form: ● Format: PDF.
Late Penalties	10% deduction of the available mark per calendar day or part thereof for up to one week. Submissions more than 7 calendar days after the due date will receive a mark of zero (0) and no assessment feedback will be provided.

---

1. Project Description

The project description provides an overview of the background and essential requirements for the cybersecurity-related projects.

This project involves developing a comprehensive machine-learning solution that integrates project management, design elements, and technical implementation to address real-world challenges. Students will work in teams to complete the project in three phases: creating a detailed project management plan, implementing a machine learning model, and developing a dynamic website to showcase their results.

The Machine Learning Web Application aims to deliver an interactive platform for users to engage with machine learning models and visualize data insights. The primary goal is to demonstrate practical machine learning applications in real-world scenarios, enhancing user interaction and understanding of the underlying models.

Core Functional Requirements for the whole project:

- Select a project topic and find relevant datasets suitable for the chosen topic.
- Investigate and analyze the topic, using machine learning techniques for prediction, attribution, or classification to gain a better understanding and response to the topic.
- **Develop a website application that allows users to interact with trained models and provides corresponding responses based on user inputs.**
- **Include data visualization features to help users understand the dataset.**

(***Note that we have highlighted any information related to Assignment 3.***)

---

2. Overview of the tasks

1. Front-End Development (React.js):

- Create a user input form with appropriate validation mechanisms.
- Implement **at least two** types of data visualization charts using D3.js, Chart.js, or Plotly.js (three charts for HD-level work).
- Design a user-friendly, responsive interface.
- Ensure seamless interaction with the back-end API for data submission and retrieval.
- Apply UI/UX principles to enhance usability and aesthetics.

2. Back-End Development (FastAPI):

- Set up a FastAPI server and define necessary routes (using **at least two** HTTP methods: GET, PUT, POST, or DELETE).
- Integrate the AI model from Assignment 2 for prediction processing.
- Develop API endpoints for handling user data and returning model predictions.
- Implement robust error handling and exception management for stable operation.

3. AI Model Integration:

- Ensure efficient execution of the AI model on the server.
- Implement data preprocessing and postprocessing logic to handle inputs and outputs.

4. Documentation:

- Write a comprehensive README file with instructions for setup and running the application.
- Prepare a comprehensive project report that covers all aspects of the system, including the system architecture, implementation details, introduction, and API documentation.

- The implementation details should provide a clear explanation of how the front-end and back-end components are developed, how the AI model is integrated, and any key design decisions made.
- Include descriptions of data preprocessing, postprocessing, error handling, and validation techniques.

Note: To achieve a **High Distinction (HD)**, students must include additional features such as at least three types of charts, interactive visualizations, and at least three advanced functionalities based on their AI model outcomes. Examples include:

1. **Enhanced Visualizations**: Implement at least three chart types with interactive features (e.g., zoom, filtering) using D3.js or Plotly.js.
2. **Real-Time Updates**: Integrate live data feeds or enable auto-refreshing charts as new predictions are generated.
3. **Custom User Input**: Provide advanced input forms with comprehensive validation and dynamic user feedback.
4. **Error Handling**: Implement robust error handling with clear, informative messages on both the front-end and back-end.
5. **UX Enhancements**: Include features such as export functionality for predictions and visualizations, but not limited to that.

---

3. Deliverables

Your submission should contain the following:

1. Project Report (PDF):

- **Project Title**: Title of your full-stack web application project (include on the cover page).
- **Group Information**: Group name and ID, student names and IDs, tutor's name (include on the cover page).
- **System Architecture**: Describe the overall architecture, including front-end, back-end, and AI model components.
- **Front-End Implementation**:
    - Briefly describe key features like the user input form and data visualization with validation.
    - Explain how data is sent to the back-end and predictions are retrieved using HTTP requests (e.g., Axios).
- **Back-End Implementation**:
    - Explain the FastAPI server setup.

    - Provide API endpoint documentation (including request type, JSON response, etc.).
- **AI Model Integration**:
    - Briefly recap the AI model from Assignment 2.
    - Explain how the model is integrated into the web application and describe any modifications for web deployment.
- **Conclusion**:
    - Explain how the solution addressed the chosen real-world issue and provided practical insights to users.
    - Identify areas for improvement and outline future enhancements like expanded functionality.
- **Bibliography**: List references in Harvard style.
- **Appendix**: Include additional files or information if necessary.

2. Source Code (ZIP file):

- **Front-End**: React.js code (exclude node_modules).
- **Back-End**: Back-end FastAPI code (include the AI model code used in the web application with any modification for integration)
- **README File**: Provide setup instructions, necessary libraries, running instructions for both front-end and back-end, and configuration steps for AI model integration.

3. Demonstration Video (Maximum 7 minutes):

- **Content**:
    - ➢ Showcase the FULL functionality of the web application.
    - ➢ Demonstrate **every aspect** of the website, including **all user interactions** (e.g., clickable buttons, input fields).
    - ➢ Ensure that **every part** of the application is shown, covering all features and components.
    - ➢ Show responsiveness on **desktop, tablet**, and **mobile** devices.
    - ➢ Walk through the **entire** workflow, from user input to data processing and result visualization.
    - ➢ Explain key technical decisions and challenges.
- **Format**: MP4 or AVI.
- **YouTube Option**:
    - ➢ If the file size exceeds the Canvas limit, upload the video as an unlisted YouTube video and provide the link in your submission. Ensure the link remains active until grading is complete.
    - ➢ If submitting a YouTube link, please include it on the cover page of your project report and also in a separate text file named group-name-video-link.txt.

- **Important**: If the video is inaccessible at the time of grading, you will receive **zero marks for most components of this assignment**, as the video serves as the primary method for presenting and demonstrating your work. Ensure that the video is properly uploaded and accessible.

4. Meeting Minutes (PDF):

- Students are required to hold at least one meeting each week since the team's inception and submit all meeting minutes, along with other deliverables, as part of the assignment. You can use the attached meeting minutes as they are or as a reference to create your own. **Meeting Minutes Example.docx**

5. Contribution Form (PDF):

- A form includes sections for the personal information of each team member, details of the contribution, and other additional information. You can download the form:
  - ➢ Group Assessment Contribution Form.docx
    (Download Group Assessment Contribution Form.docx)

---

4. Submission

You must submit your assignment via the "Assignment 3 Submission" link on the Canvas site by the deadline (Tuesday 5:00 pm (17:00) *UTC +7*, 10th April 2026, Week 12 or 13). No hard copy submission is required.

Only one student per group needs to submit all the required documents.

File Naming: All files should be named using your group name, e.g., "*session-xx-group-1-AssignmentName.pdf*". Marks may be deducted if naming and file requirements are not strictly followed. Submissions via email will not be accepted.

Detailed Submission Requirements:

1. Project Report:

- **Length**: No more than 8 pages (*excluding cover page, table of contents, bibliography, and appendix*).
- **Format**: PDF.
- **Note**: If the API documentation is too long to fit, include the most important APIs. Document at least 4 APIs in proper format, and mentioning that not all APIs are shown due to page limitations.

2. Source Code (ZIP File):

- **Structure**: Both front-end (React.js) and back-end (FastAPI) code, organized in separate folders.
- **AI Model**: The back-end folder must include the packaged AI model used in the application.
- **README File:** Markdown (.md) or PDF (.pdf) format.

3. Demonstration Video:

- **Length**: Maximum 7 minutes.
- **Format**: MP4 or AVI.
- **YouTube Option**: If the file size exceeds Canvas limits, upload the video as an unlisted YouTube video and provide the link. Ensure the link remains active until grading is complete.

4. Meeting Minutes:

- **Format**: PDF

5. Contribution Form:

- **Format**: PDF

---

5. Marking Criteria

You must acknowledge all statements and information taken from other sources and adhere to the guidelines published regarding plagiarism. All ideas and material taken from references must be cited within the report itself and a full reference list and bibliography (if appropriate) must be provided at the end of the report. Diagrams and/or tables may be used if you think this will strengthen your arguments. Remember that diagrams and tables adapted from other sources must be cited (***Harvard*** style) as well.

- **Note**: If the video is inaccessible at the time of grading, you will receive **zero marks for most components of this assignment**, as the video serves as the primary method for presenting and demonstrating your work. Ensure that the video is properly uploaded and accessible.

Important Notes:

- Do not publicly post any part of your work or reasoning on Canvas discussions to avoid violating plagiarism/collusion rules.

- Generative AI tools must not be used to generate content for this assignment.
  - However, their use for learning, idea generation, or debugging support is permitted.
  - You are responsible for understanding, writing, and being able to explain all core logic in your application.
  - Copying entire functions or components without full comprehension constitutes plagiarism.
  - All use of generative AI must be acknowledged and documented in your report.
- Late video submissions or modifications after the deadline will result in penalties.
- Ensure the video clearly demonstrates all aspects of your application for grading.

Criteria

COS30049 Assignment 3 Rubric: COS30049 (CT Innovation Project) - TechNova